

Chapter 6: Deadlocks

Questions and Answers

Question 1

Give an example of a deadlock taken from politics.

Answer 1

A political convention can deadlock if no candidate has a majority and no delegate is willing to change votes. Each candidate holds some votes but needs more votes to finish the nomination process. A parliament can also deadlock if parties cannot form a majority to pass a budget.

Question 2

Students working at individual PCs in a computer laboratory send their files to be printed by a server that spools the files on its hard disk. Under what conditions may a deadlock occur if the disk space for the print spool is limited? How may the deadlock be avoided?

Answer 2

Deadlock can occur when the spool disk fills with incomplete jobs, and each job needs more disk blocks before it can print. For example, ten partially spooled 2-MB jobs can fill a 10-MB spool area before any one job is complete.

Avoid it by allowing a job to start printing before it is fully spooled and reserving the freed spool space for that same job. Then at least one job can finish and release space.

Question 3

In the preceding question, which resources are preemptable and which are nonpreemptable?

Answer 3

The printer is nonpreemptable because a partially printed job cannot usually be interrupted and resumed cleanly by another job. The spool disk space is preemptable because the system can delete an incomplete spool file and ask the user to submit it again, assuming the protocol permits this.

Question 4

```
typedef int semaphore;  
semaphore resource_1;
```

```
void process_A(void) {  
    down(&resource_1);  
    use_resource_1();  
    up(&resource_1);  
}
```

(a)

```
typedef int semaphore;  
semaphore resource_1;  
semaphore resource_2;
```

```
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources();  
    up(&resource_2);  
    up(&resource_1);  
}
```

(b)

Figure 6-1. Using a semaphore to protect resources. (a) One resource. (b) Two resources.

In Fig. 6-1 the resources are returned in the reverse order of their acquisition. Would giving them back in the other order be just as good?

Answer 4

Yes. The order of releasing resources does not matter for that example. What matters is that the resources are eventually released so waiting processes can continue.

Question 5

The four conditions (mutual exclusion, hold and wait, no preemption and circular wait) are necessary for a resource deadlock to occur. Give an example to show that these conditions are not sufficient for a resource deadlock to occur. When are these conditions sufficient for a resource deadlock to occur?

Answer 5

They are not sufficient when resource types have multiple instances. Example: resources R and S , with one R and two S 's. Let A hold R , B hold one S , and C hold the other S . If B requests R and A requests S , the four conditions can hold, but C can finish and release S , allowing A , then B , to finish.

Key Point

The four conditions are sufficient when there is only one instance of each resource type.

Question 6

City streets are vulnerable to a circular blocking condition called gridlock, in which intersections are blocked by cars that then block cars behind them that then block the cars that are trying to enter the previous intersection, etc. All intersections around a city block are filled with vehicles that block the oncoming traffic in a circular manner. Gridlock is a resource deadlock and a problem in competition synchronization. New York City's prevention algorithm, called "don't block the box," prohibits cars from entering an intersection unless the space following the intersection is also available. Which prevention algorithm is this? Can you provide any other prevention algorithms for gridlock?

Answer 6

It is a preallocation strategy that attacks the **hold-and-wait** condition. A car may enter the intersection only if it can also reserve enough space after the intersection. Other prevention methods include traffic throttling, changing signal timing to reduce congestion, or temporarily redirecting cars so intersections can drain.

Question 7

Suppose four cars each approach an intersection from four different directions simultaneously. Each corner of the intersection has a stop sign. Assume that traffic regulations require that when two cars approach adjacent stop signs at the same time, the car on the left must yield to the car on the right. Thus, as four cars each drive up to their individual stop signs, each waits (indefinitely) for the car on the left to proceed. Is this anomaly a communication deadlock? Is it a resource deadlock?

Answer 7

It is neither a communication deadlock nor a resource deadlock. No car is holding a resource needed by another, and the cars are not waiting for messages. It is better described as a competition-synchronization or scheduling deadlock. A traffic officer choosing one car to proceed can break it.

Question 8

Is it possible that a resource deadlock involves multiple units of one type and a single unit of another? If so, give an example.

Answer 8

Yes. One process can hold some units of resource R and wait for single resource S , while another holds S and waits for more units of R . If no process can release R , the system is deadlocked.

Example: a process holds a locked data item and needs more nonpreemptable memory; another process holds the available memory and waits for the data item.

Question 9

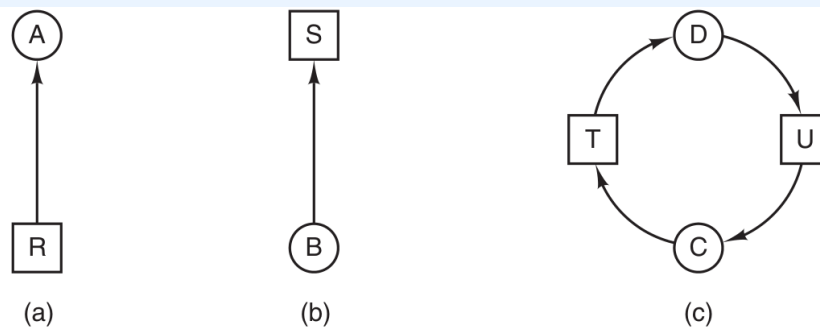


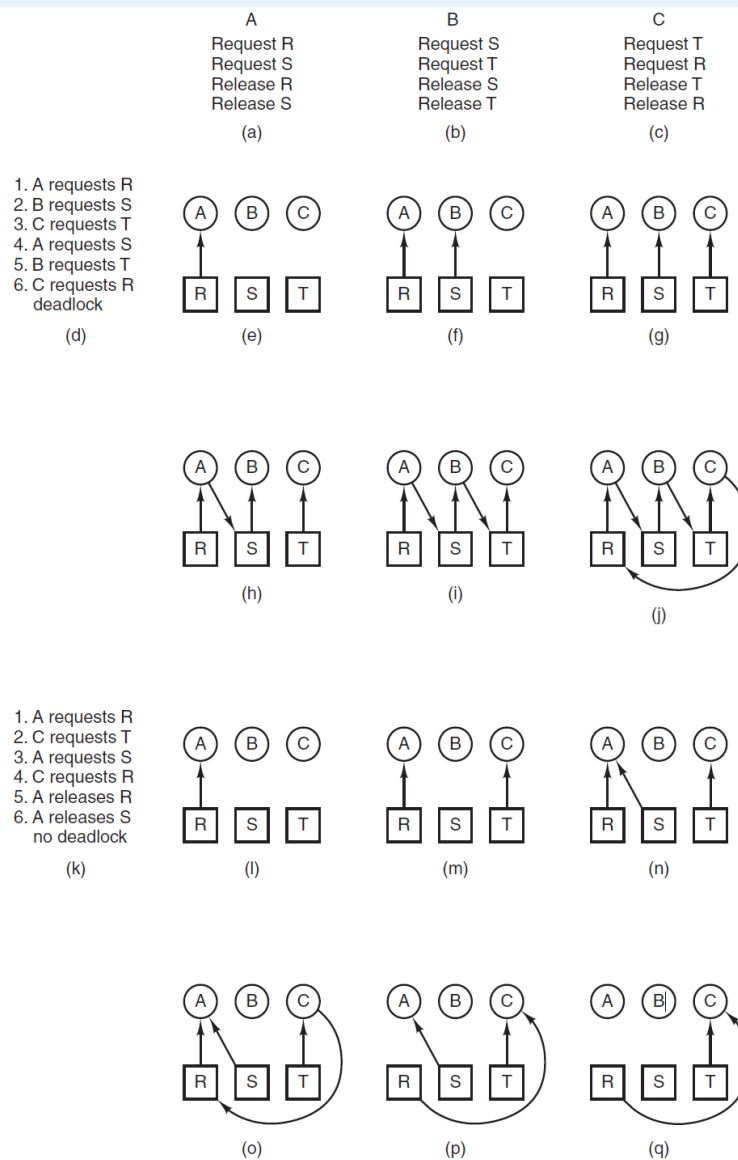
Figure 6-3. Resource allocation graphs. (a) Holding a resource. (b) Requesting a resource. (c) Deadlock.

Fig. 6-3 shows the concept of a resource graph. Do illegal graphs exist, that is, graphs that structurally violate the model we have used of resource usage? If so, give an example of one.

Answer 9

Yes. In the basic resource-allocation graph model, a single-instance resource can be assigned to only one process. A graph is illegal if one resource square has assignment arcs to two different process circles. Arcs from resource to resource or process to process also violate the model.

Question 10



Consider Fig. 6-4. Suppose that in step (o) *C* requested *S* instead of requesting *R*. Would this lead to deadlock? Suppose that it requested both *S* and *R*.

Answer 10

No. Neither change leads to deadlock because the resulting graph has no circular wait.

Question 11

Suppose that there is a resource deadlock in a system. Give an example to show that the set of processes deadlocked can include processes that are not in the circular chain in the corresponding resource allocation graph.

Answer 11

Yes. Suppose A waits for R held by B , and B waits for S held by A . That is a circular chain. If C waits for R held by A , then C is also deadlocked, even though it is not part of the cycle.

Question 12

In order to control traffic, a network router, A periodically sends a message to its neighbor, B , telling it to increase or decrease the number of packets that it can handle. At some point in time, Router A is flooded with traffic and sends B a message telling it to cease sending traffic. It does this by specifying that the number of bytes B may send (A 's window size) is 0. As traffic surges decrease, A sends a new message, telling B to restart transmission. It does this by increasing the window size from 0 to a positive number. That message is lost. As described, neither side will ever transmit. What type of deadlock is this?

Answer 12

This is a **communication deadlock**. It can be controlled by timeouts and retransmission. The window update should use absolute values or sequence numbers so duplicate messages do not cause incorrect state.

Question 13

The discussion of the ostrich algorithm mentions the possibility of process-table slots or other system tables filling up. Can you suggest a way to enable a system administrator to recover from such a situation?

Answer 13

Reserve a small portion of critical system resources for administrator-owned processes. This lets the administrator still log in, inspect the system, and kill or restart offending processes.

Question 14

Consider the following state of a system with four processes, P_1 , P_2 , P_3 , and P_4 , and five types of resources, RS_1 , RS_2 , RS_3 , RS_4 , and RS_5 :

$$E = (2 \ 4 \ 1 \ 4 \ 4) \quad A = (0 \ 1 \ 0 \ 2 \ 1)$$

$$C = \begin{bmatrix} 0 & 0 & 1 & 2 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad R = \begin{bmatrix} 1 & 1 & 0 & 2 & 1 \\ 0 & 1 & 0 & 2 & 1 \\ 0 & 2 & 0 & 3 & 1 \\ 0 & 2 & 1 & 1 & 0 \end{bmatrix}$$

Using the deadlock detection algorithm described in Section 6.4.2, show that there is a deadlock in the system. Identify the processes that are deadlocked.

Answer 14

The detection algorithm marks processes whose outstanding requests can be satisfied by the available vector. In the given state, P_2 can finish first, then P_3 can finish. After that, P_1 and P_4 still cannot be marked.

The deadlocked processes are P_1 and P_4 .

Question 15

Explain how the system can recover from the deadlock in previous problem using

- (a) recovery through preemption.
- (b) recovery through rollback.
- (c) recovery through killing processes.

Answer 15

Method	Recovery idea
Preemption	Force P_1 to give up one unit of RS_3 , allowing P_4 to finish, then P_1 .
Rollback	Roll P_1 back to a checkpoint before it acquired RS_3 .
Killing	Kill P_1 , releasing its resources.

Question 16

Suppose that in Fig. 6-6 $C_{ij} + R_{ij} > E_j$ for some i . What implications does this have for the system?

Answer 16

The process is asking for more units of a resource than the system has in total. It can never finish, even if no other process wants resources.

Question 17

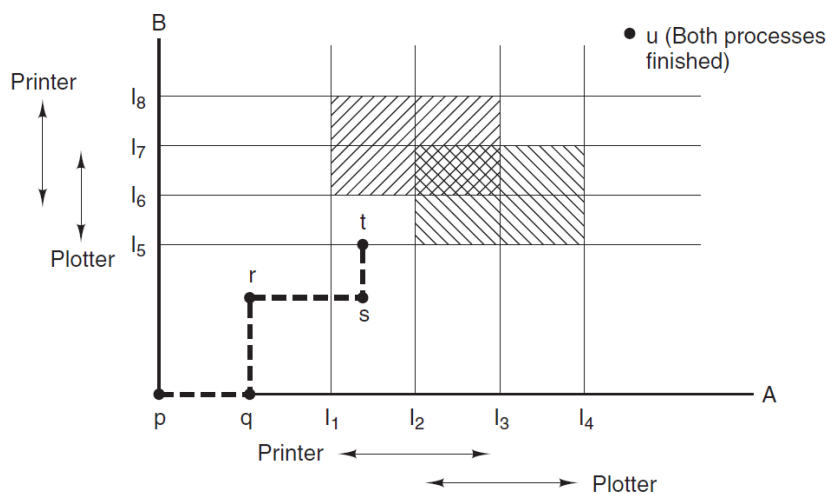


Figure 6-8. Two process resource trajectories.

All the trajectories in Fig. 6-8 are horizontal or vertical. Can you envision any circumstances in which diagonal trajectories are also possible?

Answer 17

Yes, if two or more CPUs allow multiple processes to execute simultaneously. A diagonal move means more than one process advances at the same time.

Question 18

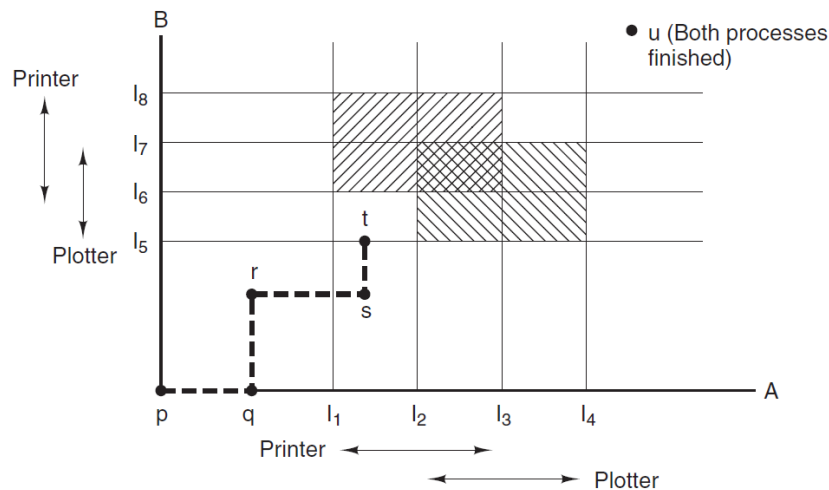


Figure 6-8. Two process resource trajectories.

Can the resource trajectory scheme of Fig. 6-8 also be used to illustrate the problem of deadlocks with three processes and three resources? If so, how can this be done? If not, why not?

Answer 18

Yes, but the graph becomes three-dimensional. The third axis represents the progress of the third process.

Question 19

In theory, resource trajectory graphs could be used to avoid deadlocks. By clever scheduling, the operating system could avoid unsafe regions. Is there a practical way of actually doing this?

Answer 19

Usually no. They require knowing in advance the exact future points at which processes will request and release resources. Real operating systems rarely have this information.

Question 20

Can a system be in a state that is neither deadlocked nor safe? If so, give an example. If not, prove that all states are either deadlocked or safe.

Answer 20

Yes. A state may not be deadlocked now, because some actions are still possible, but it may be unsafe because certain future requests can lead to deadlock.

Distinction

Deadlocked means no progress is possible now. Unsafe means no guaranteed completion sequence exists.

For example, suppose the four resource classes are tapes, plotters, scanners, and CD-ROMs, and three processes are competing for them:

Process	Has	Needs	Available
<i>A</i>	2 0 0 0	1 0 2 0	0 1 2 1
<i>B</i>	1 0 0 0	0 1 3 1	
<i>C</i>	0 1 2 1	1 0 1 0	

This state is not deadlocked because actions can still occur; for instance, *A* can still get two scanners. However, if each process asks for its remaining requirements, the system can move into a deadlocked state. Thus the state is unsafe but not currently deadlocked.

Question 21

Has Max		
A	0	6
B	0	5
C	0	4
D	0	7
Free: 10		
(a)		

Has Max		
A	1	6
B	1	5
C	2	4
D	4	7
Free: 2		
(b)		

Has Max		
A	1	6
B	2	5
C	2	4
D	4	7
Free: 1		
(c)		

Figure 6-11. Three resource allocation states: (a) Safe. (b) Safe. (c) Unsafe.

Take a careful look at Fig. 6-11(b). If *D* asks for one more unit, does this lead to a safe state or an unsafe one? What if the request came from *C* instead of *D*?

Answer 21

A request from *D* leads to an unsafe state. A request from *C* leads to a safe state.

Question 22

A system has two processes and three identical resources. Each process needs a maximum of two resources. Is deadlock possible? Explain your answer.

Answer 22

No. If both processes each hold one resource, one resource remains free. Either process can get the free resource, finish, and release its resources. Therefore deadlock cannot occur.

Question 23

Consider the previous problem again, but now with p processes each needing a maximum of m resources and a total of r resources available. What condition must hold to make the system deadlock free?

Answer 23

The worst nonfinished case is that every process holds $m - 1$ resources and needs one more. If one additional resource is available, one process can finish and release its resources.

$$r \geq p(m - 1) + 1$$

Question 24

Process	Tape drives	Plotters	Printers	Blu-rays
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resources assigned

Process	Tape drives	Plotters	Printers	Blu-rays
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still assigned

$E = (6342)$
 $P = (5322)$
 $A = (1020)$

Figure 6-12. The banker's algorithm with multiple resources.

Suppose that process A in Fig. 6-12 requests the last tape drive. Does this action lead to a deadlock?

Answer 24

No. Another process, D , can still finish. When D returns its resources, other processes such as E or A can finish, and the system can continue.

Question 25

The banker's algorithm is being run in a system with m resource classes and n processes. In the limit of large m and n , the number of operations that must be performed to check a state for safety is proportional to $m^a n^b$. What are the values of a and b ?

Answer 25

Comparing one process request row with the available vector costs $O(m)$. Finding one process that can finish may require scanning $O(n)$ rows, and this can happen for n processes.

$$O(mn^2)$$

$$a = 1, \quad b = 2$$

Question 26

A system has four processes and five allocatable resources. The current allocation and maximum needs are as follows:

Process	Allocated	Maximum	Available
A	10211	11213	00x11
B	20110	22210	
C	11010	21310	
D	11110	11221	

What is the smallest value of x for which this is a safe state?

Answer 26

The need matrix is:

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 2 \\ 0 & 2 & 1 & 0 & 0 \\ 1 & 0 & 3 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

If $x = 0$, the system is immediately deadlocked. If $x = 1$, D can finish, but the remaining state is still deadlocked. If $x = 2$, D can finish, then C , then B , then A .

$$x = 2$$

Question 27

One way to eliminate circular wait is to have rule saying that a process is entitled only to a single resource at any moment. Give an example to show that this restriction is unacceptable in many cases.

Answer 27

Many tasks require multiple resources at once. For example, copying a large file from tape to printer may require holding the tape drive, memory buffer, and printer in a coordinated way. Releasing each resource after every small step increases execution time and may break output continuity.

Question 28

Two processes, A and B , each need three records, 1, 2, and 3, in a database. If A asks for them in the order 1, 2, 3, and B asks for them in the same order, deadlock is not possible. However, if B asks for them in the order 3, 2, 1, then deadlock is possible. With three resources, there are $3!$ or six possible combinations in which each process can request them. What fraction of all the combinations is guaranteed to be deadlock free?

Answer 28

If B requests record 1 first, one process gets it and the other blocks, so no deadlock occurs. The safe orders are:

$$123, \quad 132$$

The other four orders can lead to deadlock.

$$\frac{2}{6} = \frac{1}{3} \text{ are guaranteed deadlock free}$$

Question 29

A distributed system using mailboxes has two IPC primitives, send and receive. The latter primitive specifies a process to receive from and blocks if no message from that process is available, even though messages may be waiting from other processes. There are no shared resources, but processes need to communicate frequently about other matters. Is deadlock possible? Discuss.

Answer 29

Yes. Example: *A* sends to *B* and waits for *B*, *B* sends to *C* and waits for *C*, and *C* sends to *A* and waits for *A*. If each is waiting for a specific reply, the system has a communication deadlock.

Question 30

In an electronic funds transfer system, there are hundreds of identical processes that work as follows. Each process reads an input line specifying an amount of money, the account to be credited, and the account to be debited. Then it locks both accounts and transfers the money, releasing the locks when done. With many processes running in parallel, there is a very real danger that a process having locked account *x* will be unable to lock *y* because *y* has been locked by a process now waiting for *x*. Devise a scheme that avoids deadlocks. Do not release an account record until you have completed the transactions. (In other words, solutions that lock one account and then release it immediately if the other is locked are not allowed.)

Answer 30

Order all account records by account number. Each process locks the lower-numbered account first and then the higher-numbered account.

Why It Works

No process ever waits for a lower-numbered account while holding a higher-numbered one, so circular wait is impossible.

Question 31

One way to prevent deadlocks is to eliminate the hold-and-wait condition. In the text it was proposed that before asking for a new resource, a process must first release whatever resources it already holds (assuming that is possible). However, doing so introduces the danger that it may get the new resource but lose some of the existing ones to competing processes. Propose an improvement to this scheme.

Answer 31

Change the request semantics: if the new resource is available, grant it and let the process keep its old resources. If the new resource is not available, release all current resources. This avoids the case where the process gets the new resource but loses old ones.

Question 32

A computer science student assigned to work on deadlocks thinks of the following brilliant way to eliminate deadlocks. When a process requests a resource, it specifies a time limit. If the process blocks because the resource is not available, a timer is started. If the time limit is exceeded, the process is released and allowed to run again. If you were the professor, what

grade would you give this proposal and why?

Answer 32

No. The process still needs the resource, so it will likely request it again and block again. This can even make things worse by causing the process to lose waiting priority.

Grade

This proposal deserves a failing grade as a deadlock solution because it does not remove the cause of the deadlock.

Question 33

Main memory units are preempted in swapping and virtual memory systems. The processor is preempted in time-sharing environments. Do you think that these preemption methods were developed to handle resource deadlock or for other purposes? How high is their overhead?

Answer 33

No. Virtual memory and time sharing were mainly developed to improve usability, sharing, isolation, and abstraction. They can help with deadlock-related resource issues, but that was not their only purpose. Their overhead is significant: context switches, interrupts, MMU updates, caches, and specialized hardware are required.

Question 34

Explain the differences between deadlock, livelock, and starvation.

Answer 34

Term	Meaning
Deadlock	Processes are blocked forever, each waiting for an event only another blocked process can cause.
Livelock	Processes are not blocked, but keep taking actions that prevent progress.
Starvation	A process waits indefinitely because other processes keep getting priority or resources first.

Question 35

Assume two processes are issuing a seek command to reposition the mechanism to access the disk and enable a read command. Each process is interrupted before executing its read, and discovers that the other has moved the disk arm. Each then reissues the seek command, but is again interrupted by the other. This sequence continually repeats. Is this a resource deadlock or a livelock? What methods would you recommend to handle the anomaly?

Answer 35

It is livelock, not resource deadlock. The processes are active and keep retrying, but make no progress. It can be handled by preallocating the disk-arm operation, using a higher-level disk scheduler, or adding randomized backoff/timeouts.

Question 36

Local Area Networks utilize a media access method called CSMA/CD, in which stations sharing a bus can sense the medium and detect transmissions as well as collisions. In the Ethernet protocol, stations requesting the shared channel do not transmit frames if they sense the medium is busy. When such transmission has terminated, waiting stations each transmit their frames. Two frames that are transmitted at the same time will collide. If stations immediately and repeatedly retransmit after collision detection, they will continue to collide indefinitely.

- (a) Is this a resource deadlock or a livelock?
- (b) Can you suggest a solution to this anomaly?
- (c) Can starvation occur with this scenario?

Answer 36

It is livelock: stations keep acting but make no progress. The standard solution is randomized exponential backoff. After a collision, each station waits a random number of time slots; after repeated collisions, the waiting range increases.

Starvation can occur because access is probabilistic: a station may repeatedly lose while newer stations succeed.

Question 37

A program contains an error in the order of cooperation and competition mechanisms, resulting in a consumer process locking a mutex (mutual exclusion semaphore) before it blocks on an empty buffer. The producer process blocks on the mutex before it can place a value in the empty buffer and awaken the consumer. Thus, both processes are blocked forever, the producer waiting for the mutex to be unlocked and the consumer waiting for a signal from the producer. Is this a resource deadlock or a communication deadlock? Suggest methods for its control.

Answer 37

It is best classified as a communication deadlock caused by incorrect ordering of synchronization operations. The producer waits for a mutex held by the consumer, while the consumer waits for a signal only the producer can generate.

Controls include writing the synchronization order correctly, using monitors, or using timeouts that release the consumer's mutex.

Question 38

Cinderella and the Prince are getting divorced. To divide their property, they have agreed on the following algorithm. Every morning, each one may send a letter to the other's lawyer requesting one item of property. Since it takes a day for letters to be delivered, they have agreed that if both discover that they have requested the same item on the same day, the next day they will send a letter canceling the request. Among their property is their dog, Woof, Woof's doghouse, their canary, Tweeter, and Tweeter's cage. The animals love their houses, so it has been agreed that any division of property separating an animal from its house is invalid, requiring the whole division to start over from scratch. Both Cinderella and the Prince desperately want Woof. So that they can go on (separate) vacations, each spouse has programmed a personal computer to handle the negotiation. When they come back from vacation, the computers are still negotiating. Why? Is deadlock possible? Is starvation possible? Discuss your answer.

Answer 38

If both request the same item first, they both cancel and retry forever, causing starvation. If one requests the dog and the other requests the doghouse, an invalid division can be detected and restarted, but the same pattern can repeat forever. Thus both deadlock-like repetition and starvation are possible, depending on the request sequence.

Question 39

A student majoring in anthropology and minoring in computer science has embarked on a research project to see if African baboons can be taught about deadlocks. He locates a deep canyon and fastens a rope across it, so the baboons can cross hand-overhand. Several baboons can cross at the same time, provided that they are all going in the same direction. If eastward-moving and westward-moving baboons ever get onto the rope at the same time, a deadlock will result (the baboons will get stuck in the middle) because it is impossible for one baboon to climb over another one while suspended over the canyon. If a baboon wants to cross the canyon, he must check to see that no other baboon is currently crossing in the opposite direction. Write a program using semaphores that avoids deadlock. Do not worry about a series of eastward-moving baboons holding up the westward-moving baboons indefinitely.

Answer 39

Use one mutex to protect counters and one binary semaphore for the rope.

```
semaphore mutex = 1;
semaphore rope = 1;
int east = 0, west = 0;

eastbound() {
    down(mutex);
    east++;
    if (east == 1) down(rope);
    up(mutex);

    cross_east();

    down(mutex);
    east--;
    if (east == 0) up(rope);
    up(mutex);
}

westbound() {
    down(mutex);
    west++;
    if (west == 1) down(rope);
    up(mutex);

    cross_west();

    down(mutex);
    west--;
    if (west == 0) up(rope);
    up(mutex);
}
```

The first process in a direction locks the rope; the last one releases it. This prevents mixed directions but can starve the opposite direction.

Question 40

Repeat the previous problem, but now avoid starvation. When a baboon that wants to cross to the east arrives at the rope and finds baboons crossing to the west, he waits until the rope is empty, but no more westward-moving baboons are allowed to start until at least one baboon has crossed the other way.

Answer 40

One clean solution is to use a turnstile so arriving processes cannot continuously extend the current direction while the other direction is waiting.

```
semaphore turnstile = 1;
semaphore mutex = 1;
semaphore rope = 1;
int east = 0, west = 0;

eastbound() {
    down(turnstile); up(turnstile);

    down(mutex);
    east++;
    if (east == 1) down(rope);
    up(mutex);

    cross_east();

    down(mutex);
    east--;
    if (east == 0) up(rope);
    up(mutex);
}

westbound() {
    down(turnstile); up(turnstile);

    down(mutex);
    west++;
    if (west == 1) down(rope);
    up(mutex);

    cross_west();

    down(mutex);
    west--;
    if (west == 0) up(rope);
    up(mutex);
}
```

For a full implementation, a waiting opposite-direction group should close the turnstile until the current group drains, then let the waiting group pass. The key idea is to prevent an endless stream in one direction from repeatedly joining ahead of the waiting direction.

Question 41

Program a simulation of the banker's algorithm. Your program should cycle through each of the bank clients asking for a request and evaluating whether it is safe or unsafe. Output a log of requests and decisions to a file.

Answer 41

The program should keep the standard banker's-algorithm state:

- E : total resources;
- A : available resources;
- C : current allocation;
- M : maximum demand;
- $N = M - C$: remaining need.

For each generated or input request, temporarily grant it, run the safety test, and commit it only if the resulting state is safe. Log the process, request vector, decision, and safe sequence if one exists.

Question 42

Write a program to implement the deadlock detection algorithm with multiple resources of each type. Your program should read from a file the following inputs: the number of processes, the number of resource types, the number of resources of each type in existence (vector E), the current allocation matrix C (first row, followed by the second row, and so on), the request matrix R (first row, followed by the second row, and so on). The output of your program should indicate whether there is a deadlock in the system. In case there is, the program should print out the identities of all processes that are deadlocked.

Answer 42

Read E , C , and R . Compute $A = E - \sum C_i$. Then repeatedly find an unmarked process i such that $R_i \leq A$, mark it, and add its allocation back:

$$A := A + C_i$$

When no more processes can be marked, every unmarked process is deadlocked.

Question 43

Write a program that detects if there is a deadlock in the system by using a resource allocation graph. Your program should read from a file the following inputs: the number of processes and the number of resources. For each process it should read four numbers: the number of resources it is currently holding, the IDs of resources it is holding, the number of resources it is currently requesting, the IDs of resources it is requesting. The output of program should indicate if there is a deadlock in the system. In case there is, the program should print out the identities of all processes that are deadlocked.

Answer 43

Build a directed graph with process nodes and resource nodes. Add:

- an edge from a process to a resource for each requested resource;
- an edge from a resource to a process for each held resource.

Then run cycle detection, such as depth-first search with colors. In a single-instance resource graph, a cycle indicates deadlock. Print the process nodes that belong to the cycle.

Question 44

In certain countries, when two people meet they bow to each other. The protocol is that one of them bows first and stays down until the other one bows. If they bow at the same time, they will both stay bowed forever. Write a program that does not deadlock.

Answer 44

Use a single mutex or a deterministic ordering rule so only one person can begin the bow protocol first. For example:

```
semaphore bow_mutex = 1;

void meet(person me, person other) {
    down(&bow_mutex);
    bow(me);
    wait_for_bow(other);
    stand_up(me);
    up(&bow_mutex);
}
```

The mutex serializes the protocol, so two people cannot both bow first and wait forever. A practical implementation could also use a tie-breaker, such as the lower process ID bows first, but the essential requirement is to remove the symmetric simultaneous wait.